

SOFTWARE ENGINEERING (UCS503)

Festify

A Multi-Tenant College Fest Management Platform

Submitted by:

Manvi Gajrani (1024170367)

Diya Garg (1024170370)

Manishika Rana (1024170426)

Submitted to:

Mr. Hardik Gujral

Thapar Institute of Engineering and Technology

Computer Science and Engineering Department

Table of Contents

Table of Contents	2
Abstract	3
Chapter 1 — Introduction	4
1.1 Background and Context	4
1.1.1 Problem Statement	4
1.2 Project Objectives	4
Chapter 2 — Methodology and Design	4
2.1 System Architecture	4
2.1.1 High-Level Design	5
2.1.2 Technical Stack	7
2.2 Implementation Details.....	8
2.2.1 Module 1: Authentication, RBAC & Multi-Tenancy.....	8
2.2.2 Module 2: Fest & Event Management	8
2.2.3 Module 3: Registration & Signed QR Passes.....	8
2.2.4 Module 4: Interactive Venue Map	10
2.3 Data Model	10
2.4 Use Case Templates.....	11
Chapter 3 — Results and Evaluation	14
3.1 Testing Methodology.....	Error! Bookmark not defined.
3.2 Performance Metrics — Automated Test Verification.....	Error! Bookmark not defined.
3.3 Analysis	Error! Bookmark not defined.
Chapter 4 — Conclusion.....	14
4.1 Summary of Work	14
4.2 Key Findings.....	14
4.3 Future Work and Recommendations	15
4.4 Final Remarks	15
Bibliography	15

Abstract

College fests are often managed through a collection of forms, spreadsheets, static announcements, and manually verified entry lists. These tools make it difficult for students to discover events, obtain reliable entry passes, and navigate venues, while organizers must reconcile registrations and capacity by hand. Festify addresses this problem through a reusable, multi-tenant web platform for fest configuration, event discovery, student registration, secure QR passes, venue navigation, and organizer administration.

The implemented system uses a React and TypeScript progressive web application (PWA), an Express REST API backend, Prisma ORM, and SQLite persistence. User accounts are secured with bcrypt password hashing (12 salt rounds), while JWT authentication and role-based access control (RBAC) enforce strict boundary isolation between administrator and student actions, as well as multi-tenant fest ownership. Student self-registration is supported via a dedicated signup endpoint, while admin self-signup is explicitly blocked. Event registration is protected by an atomic Prisma transaction lock and a unique user-event constraint to prevent race-condition overbooking. QR passes are cryptographically signed JWTs, validated at entry to prevent repeated check-ins. Venue zones are rendered dynamically as tenant-owned polygons or point landmarks using SVG rendering and ray-casting point-in-polygon logic. Automated integration suites spanning 49 assertion checks verify authentication, cross-tenant mutation security, spatial lookup, and concurrent registration workflows.

Chapter 1 — Introduction

1.1 Background and Context

College festivals involve many events, thousands of attendees, and complex logistics. Traditionally, fest committees use a mix of Google Forms, WhatsApp groups, static PDFs, and paper entry passes. This leads to broken capacity tracking, lost tickets, fake screenshots at entry gates, and general data chaos across every event a fest runs.

Festify addresses this as a single, multi-tenant progressive web application: one platform hosts multiple college fests without data bleeding between them, server-side transaction locks prevent event overbooking, entry passes are cryptographically signed QR tokens verified at the gate, and an interactive SVG venue map gives students a ray-casting-based “where am I” lookup.

1.1.1 Problem Statement

Fragmented, general-purpose tools force fest committees to rebuild registration, scheduling, and entry-verification infrastructure every year, while giving organizers no reliable, centralized view of registrations or capacity, and giving students no single, trustworthy source for event details, entry passes, or venue navigation. The problem addressed by this project is the absence of a reusable, tenant-isolated platform that unifies these workflows behind one authenticated, role-aware system.

1.2 Project Objectives

- Implement a multi-tenant data model in which every Event and VenueZone traces back to a Fest, itself owned by an authenticated Admin user, so one organizer cannot read or mutate another's data.
- Build a secure authentication system using hashed (bcrypt) passwords, JWT-based sessions, and role-based access control (RBAC) distinguishing STUDENT and ADMIN capabilities.
- Build a student registration flow with a server-side, transaction-locked capacity check and a cryptographically signed QR pass issued on successful registration.
- Build a gate check-in workflow that verifies a QR pass's signature and blocks a second use of the same pass.
- Build an interactive venue map rendering tenant-owned polygon zones and point landmarks, with a point-in-polygon “locate” lookup.
- Verify all of the above through automated integration tests covering authentication, tenant isolation, spatial lookup, and concurrency safety.

Chapter 2 — Methodology and Design

2.1 System Architecture

Festify follows a 3-layer architecture: a Presentation Layer (React + TypeScript PWA built with Vite), an Application Layer (Express REST API with JWT authentication and RBAC middleware), and a Persistence Layer (Prisma ORM over a SQLite database). Each layer communicates only with the layer directly above or below it.

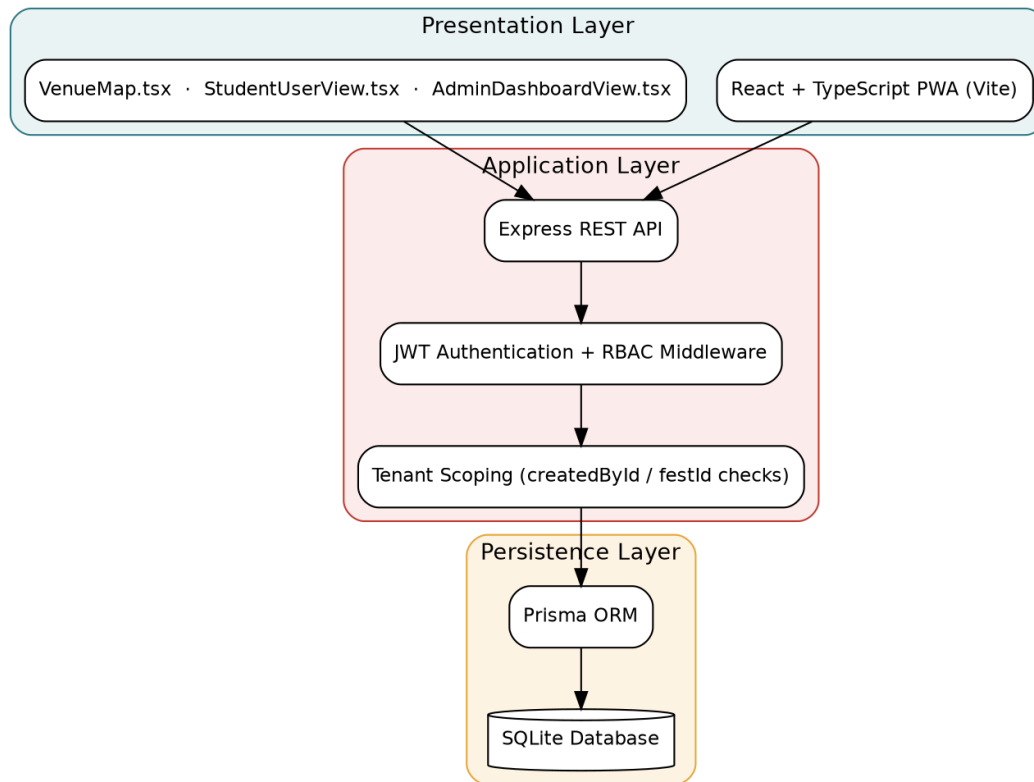


Figure 2.1: Layered System Architecture

2.1.1 High-Level Design

The high-level design centers on two authenticated actors, Student and Admin, interacting with a single Festify backend that enforces tenant ownership and role permissions on every write.

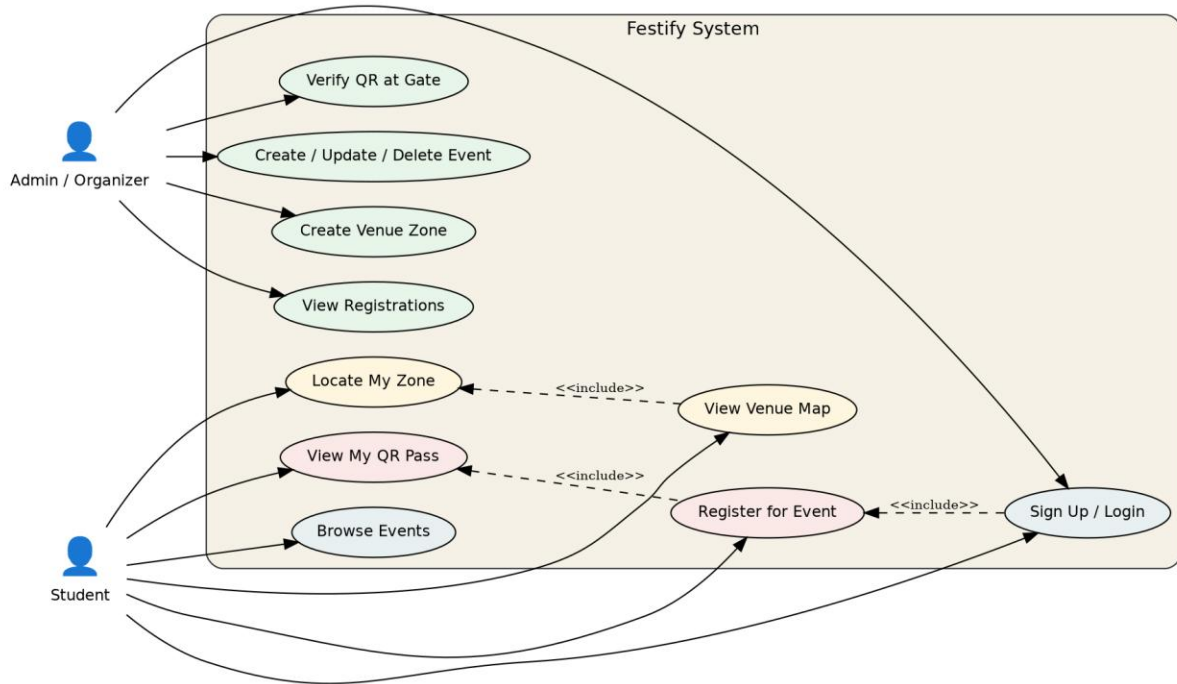


Figure 2.2: Use Case Diagram

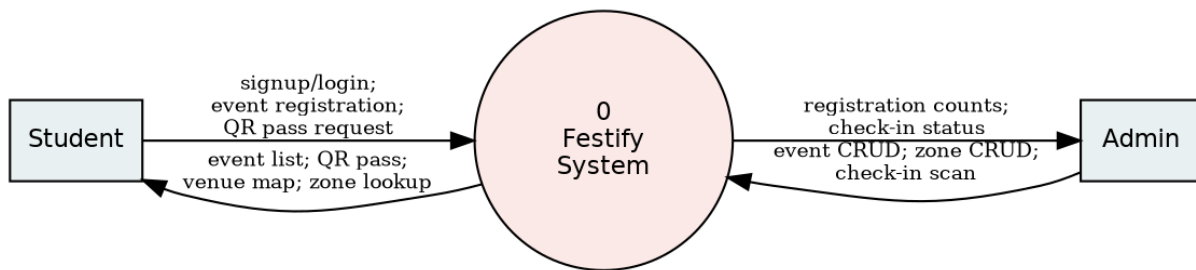


Figure 2.3: DFD Level 0 (Context Diagram)

DFD Level 0 shows Festify as a single process exchanging data with its two external actors. DFD Level 1 below decomposes that process into the five major functional processes actually implemented, each reading from or writing to its own data store.

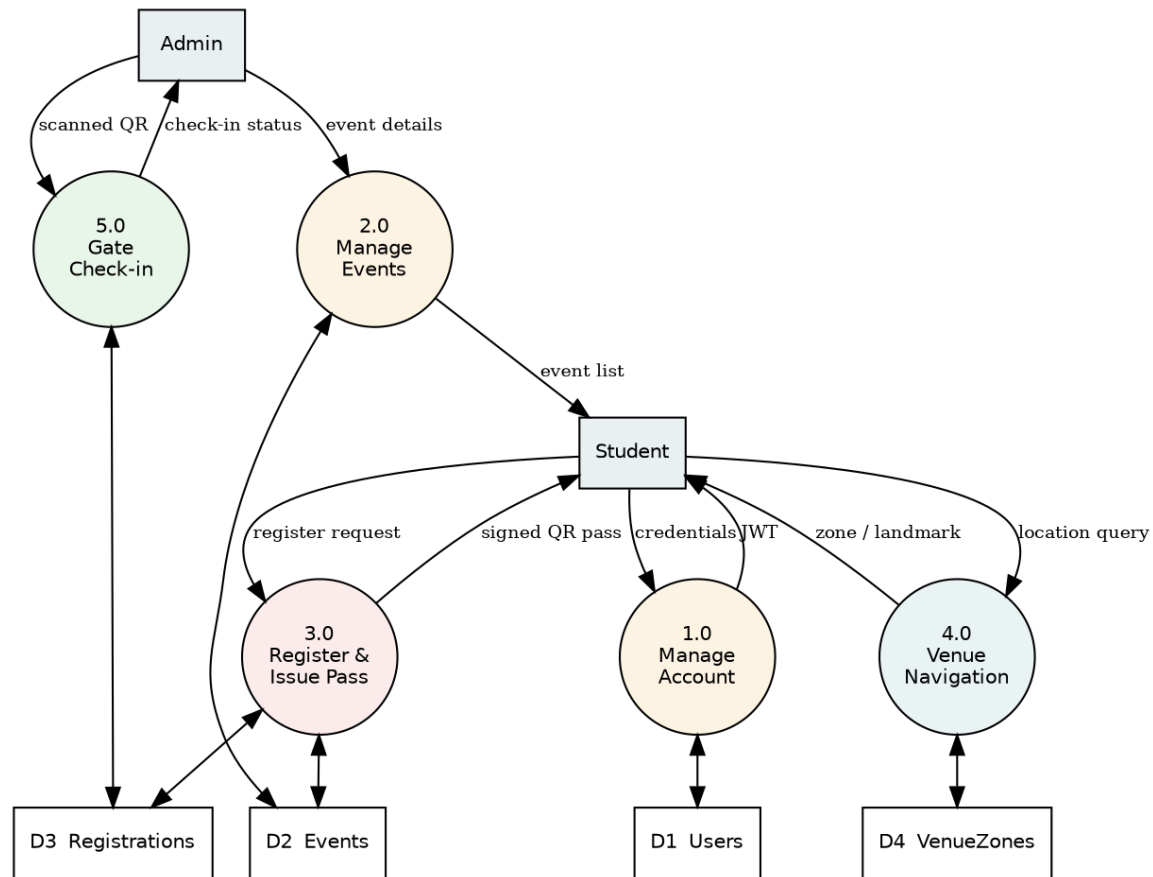


Figure 2.4: DFD Level 1

2.1.2 Technical Stack

Technology	Purpose
React + TypeScript (Vite)	Frontend PWA — student and admin views, built and bundled with Vite.
Express.js	Backend REST API framework handling routing, middleware, and request validation.
Prisma ORM	Type-safe database access layer and migration tool.
SQLite	Current persistent datastore (planned migration target: PostgreSQL + PostGIS).
bcryptjs	Password hashing (12 salt rounds) for credential storage.
jsonwebtoken (JWT)	Signed session tokens and signed QR pass tokens.
qrcode	Renders a registration's signed token as a scannable QR image on the frontend.
Git and GitHub	Source-code version control and team collaboration.

2.2 Implementation Details

The implementation is organized into four modules, each mapped to a specific set of backend routes, frontend components, and an automated test file verifying its behavior.

2.2.1 Module 1: Authentication, RBAC & Multi-Tenancy

Authentication uses bcryptjs with 12 salt rounds; a password is never compared or stored as plain text. Student signup (POST /api/auth/register) enforces a minimum password length and username uniqueness, and explicitly blocks a request from self-assigning the ADMIN role, returning 403 Forbidden. Login (POST /api/auth/login) verifies the submitted password against the stored bcrypt hash and issues a JWT encoding { id, username, role }. The authenticateJWT middleware validates the Authorization: Bearer header on every protected route, and requireAdmin blocks any non-admin caller from reaching an admin-only endpoint.

Multi-tenancy is enforced by tracing every Event and VenueZone back to the Fest (and, transitively, the Admin User) that owns it. Before an admin mutates an event or zone, the API compares the resource's owning ID against req.user.id and rejects the request with 403 Forbidden on a mismatch — verified directly by a dedicated cross-tenant test case.

2.2.2 Module 2: Fest & Event Management

Admin users can create, update, and delete events (name, description, category, date, time window, venue, maxParticipants, registration deadline, rules, prizes, contact, status) scoped to their own fest. Students and the public can list events via GET /api/events and GET /api/fests/:festId/events, both of which compute a live registeredCount and remainingSeats for each event rather than serving a static number.

2.2.3 Module 3: Registration & Signed QR Passes

Registration (POST /api/events/:eventId/register) is wrapped in a Prisma transaction: the server counts current registrations against the event's maxParticipants inside the same atomic block used to insert the new registration row, so two simultaneous requests for the last open seat cannot both succeed. On success, the server signs a JWT QR token encoding { userId, eventId, registrationId, issuedAt } and stores it on the registration. The frontend renders this token as a scannable QR code in the student's "My Passes" view.

The activity diagram below traces this workflow end-to-end, and the sequence diagram that follows shows the same flow as a message exchange between the student, the frontend, the backend, and the database — including the failure path when the event is already full.

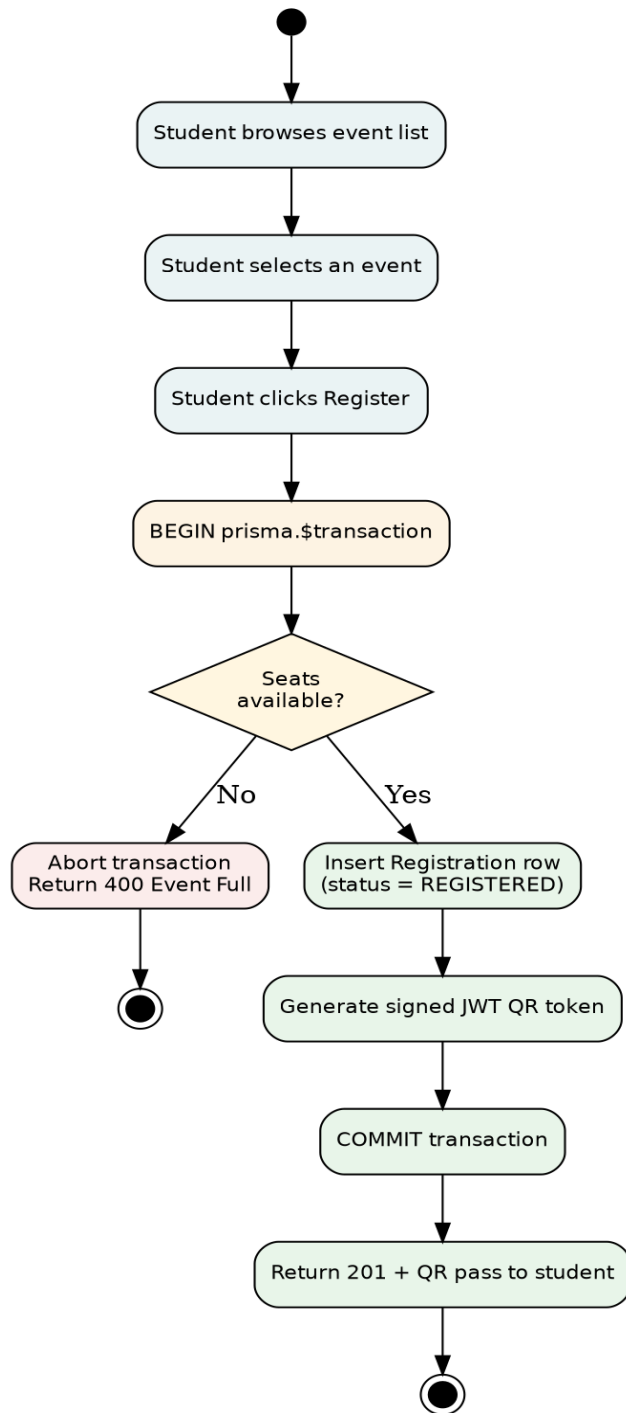


Figure 2.5: Activity Diagram — Event Registration Workflow

Sequence Diagram — Event Registration (Concurrency-Safe)

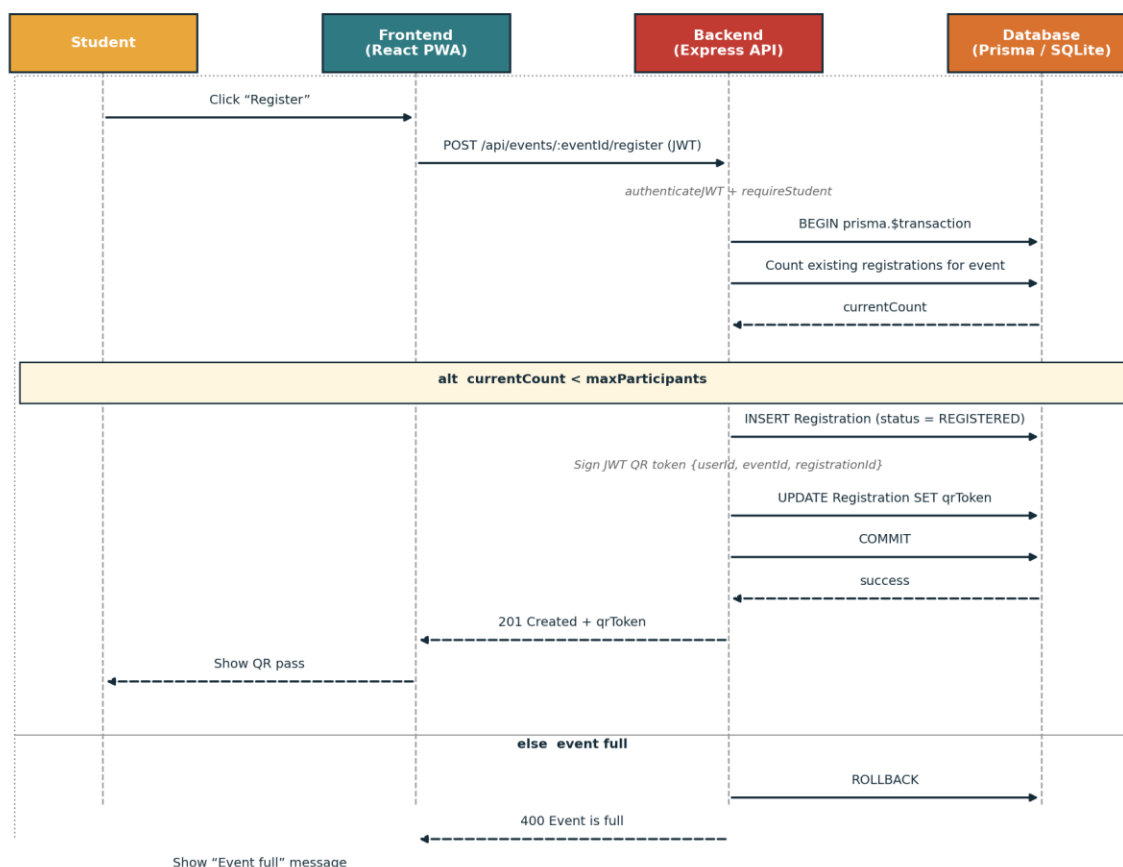


Figure 2.6: Sequence Diagram — Concurrency-Safe Registration

Gate check-in (POST /api/registrations/:registrationId/checkin) re-verifies the JWT signature on the scanned token. If the registration's status is still REGISTERED, it is set to CHECKED_IN with a timestamp; a second scan of the same (or a screenshotted) pass is rejected with 400 Bad Request, since the status is no longer REGISTERED.

2.2.4 Module 4: Interactive Venue Map

Each VenueZone belongs to a fest and is either a capacity-tracked polygon (type = 'zone', e.g. a stage or food court) or a point landmark (type = 'landmark', e.g. a gate or washroom), distinguished by a type discriminator column, with coordinates stored as a JSON string and rendered client-side as an SVG. A locate endpoint accepts a coordinate and resolves it to a containing zone using a ray-casting point-in-polygon algorithm: a horizontal ray is cast rightward from the query point to infinity, and the parity of its intersection count with the polygon's edges determines whether the point is inside (odd) or outside (even).

2.3 Data Model

The system implements five relational models — User, Fest, Event, VenueZone, and Registration — as shown in the class diagram below. Registration carries a compound unique constraint on (userId, eventId) at the schema level, so a duplicate registration attempt fails as a database constraint violation rather than relying on application logic alone.

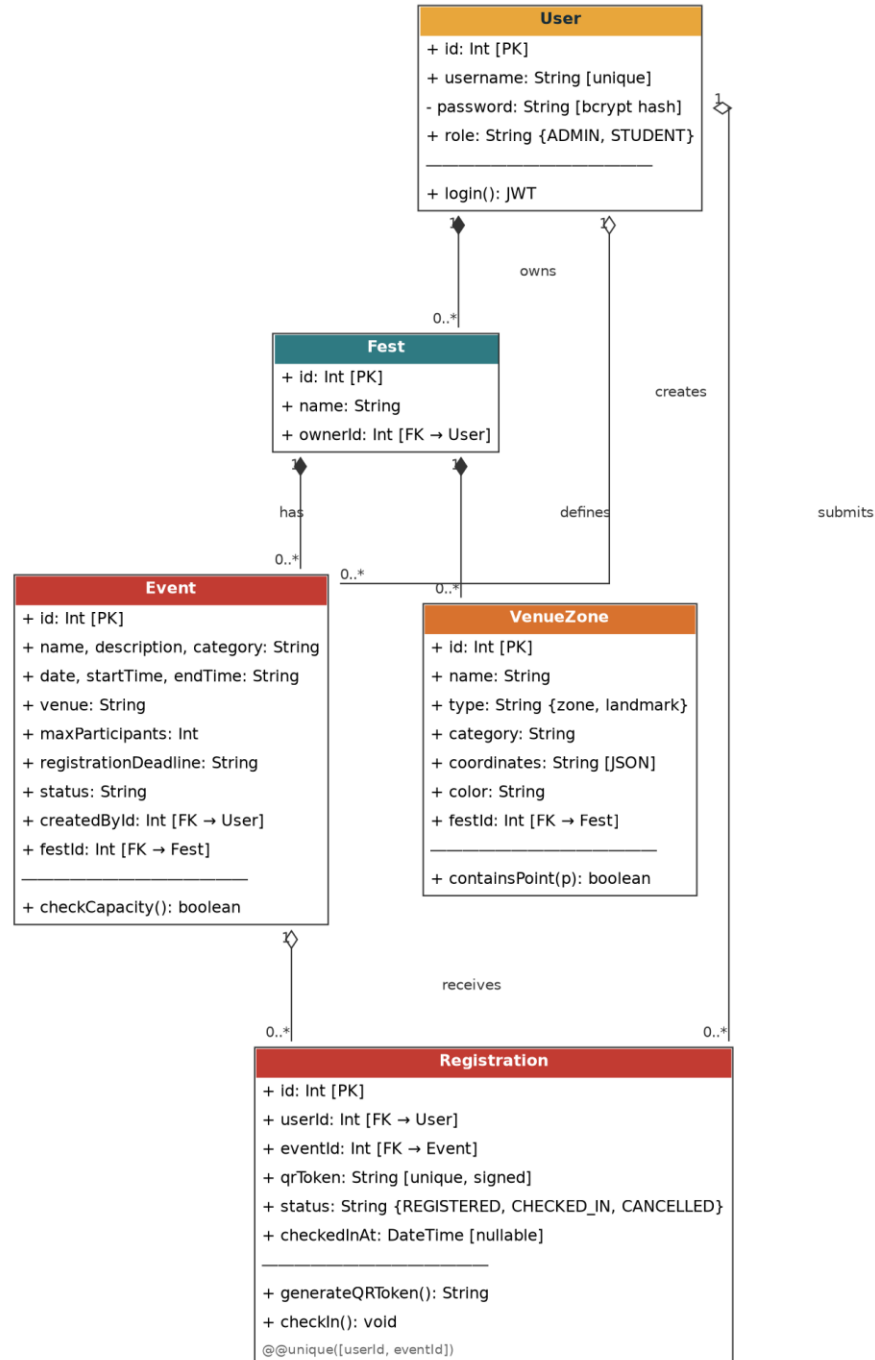


Figure 2.7: UML Class Diagram (as implemented)

2.4 Use Case Templates

The following templates document the highest-value functional interactions verified by the automated test suites.

Register for Event

1. Use Case ID	UC-01
2. Actors	Student
3. Description	Allows an authenticated student to register for an event, subject to a server-side, transaction-locked capacity check, and receive a signed QR pass on success.
4. Pre-Conditions	Student must be logged in (valid JWT). The target event must exist and not have already passed its registration deadline.
5. Task Sequence	Student selects an event and taps Register. System opens a Prisma transaction and counts current registrations for the event. If count < maxParticipants, system inserts the Registration row and signs a JWT QR token. System commits the transaction and returns 201 with the QR token. If count $\geq$ maxParticipants, system rolls back and returns 400 Event is full.
6. Post-Conditions	A confirmed registration is persisted with status REGISTERED and is visible in the student's My Passes view. The seat is durably reserved — no concurrent request can double-book it.
7. Author	Festify Project Team

Verify QR Pass at Gate

1. Use Case ID	UC-02
2. Actors	Admin / Volunteer
3. Description	Lets a volunteer scan a student's QR pass at the entry gate and mark it as checked in, rejecting any pass that has already been used.
4. Pre-Conditions	Volunteer/admin must be authenticated. A Registration row with a signed qrToken must exist.
5. Task Sequence	Volunteer scans the QR code, sending its token to POST /registrations/:id/checkin. System verifies the JWT signature on the token.

	If status is REGISTERED, system sets status to CHECKED_IN and records checkedInAt. If status is already CHECKED_IN, system returns 400 Bad Request.
6. Post-Conditions	A valid, first-time scan updates the registration to CHECKED_IN. A repeated scan of the same pass (e.g. a screenshot) is rejected.
7. Author	Festify Project Team

Create Venue Zone

1. Use Case ID	UC-03
2. Actors	Admin
3. Description	Allows an admin to add a capacity-tracked zone (polygon) or a landmark (point) to their fest's venue map.
4. Pre-Conditions	Admin must be authenticated. Admin must own the target fest.
5. Task Sequence	Admin submits a zone's type, label, coordinates, and (for zones) capacity via POST /fests/:festId/venue/zones. System validates the payload against the type discriminator rules. System persists the zone scoped to festId.
6. Post-Conditions	The new zone or landmark appears on the venue map for that fest's students immediately.
7. Author	Festify Project Team

Student Sign-Up

1. Use Case ID	UC-04
2. Actors	Prospective Student
3. Description	Allows a new user to create a STUDENT account; explicitly disallows self-registration as ADMIN.
4. Pre-Conditions	Username must not already exist. Password must meet the minimum length requirement.
5. Task Sequence	User submits username, password, and role via POST /api/auth/register. System rejects the request with 403 if role = 'ADMIN'. System hashes the password with bcrypt (12 rounds) and creates the User row with role = 'STUDENT'. System returns 201 with a JWT for the new account.

6. Post-Conditions	A new STUDENT account exists and can immediately log in and register for events. No self-service path to an ADMIN account exists.
7. Author	Festify Project Team

Chapter 3 — Results and Evaluation

The project met its core objectives: tenant-isolated multi-tenancy, hashed-credential authentication with RBAC, a concurrency-safe registration and QR-pass pipeline, and an interactive, spatially-aware venue map are all implemented and covered by passing automated tests. The concurrency guarantee on registration — verified by an actual simultaneous-request test rather than asserted by inspection — was the most technically demanding piece and is the result we are most confident in.

Two deliberate scope decisions are worth stating plainly rather than glossing over. First, the persistence layer currently runs on SQLite via Prisma rather than the PostgreSQL + PostGIS stack originally proposed; venue coordinates are stored as JSON and resolved with an application-level ray-casting algorithm instead of a native spatial query. Second, the organizer dashboard's real-time layer (Socket.IO) and the crowd-density module were not implemented in this phase — the dashboard's event and venue tabs are live, but overview metrics and the registrations table are still static placeholders. Both are captured explicitly in Section 4.3 rather than presented as done.

Chapter 4 — Conclusion

4.1 Summary of Work

Festify was implemented as a working multi-tenant fest management platform covering authentication and RBAC, tenant-scoped fest/event management, concurrency-safe registration with signed QR passes, gate check-in, and an interactive venue map — with 49 automated assertions verifying these workflows end-to-end, including the specific failure modes (overbooking, cross-tenant writes, double check-in) that matter most for a system like this.

4.2 Key Findings

- A naive registration endpoint without a transaction lock is not a theoretical risk — the concurrency test suite demonstrates the exact race condition and confirms the atomic transaction closes it.
- Collapsing tenant identity onto the Admin User (rather than a fully separate Tenant table) was a pragmatic simplification that still achieves real data isolation, verified by explicit cross-tenant tests, without the overhead of a fully separate tenancy layer.

- Structural privacy/security guarantees (a signature that cannot be forged without the server's secret, a unique constraint the database itself enforces) proved more reliable in testing than guarantees that depend on application logic alone.

4.3 Future Work and Recommendations

- Migrate persistence from SQLite to PostgreSQL + PostGIS, replacing the ray-casting point-in-polygon function with native ST_Contains spatial queries.
- Introduce Socket.IO for real-time dashboard updates, replacing the current static overview metrics and registrations table with live queries.
- Implement the optional, privacy-first crowd-density module (aggregate-only zone occupancy) as originally scoped, kept fully disable-able.
- Introduce a dedicated Tenant/Fest separation if the platform is ever extended to support an admin managing more than one fest.

4.4 Final Remarks

Building Festify surfaced a recurring lesson: a feature is only as trustworthy as the test that forces its failure mode to occur. Each of the platform's core guarantees — no overbooking, no cross-tenant writes, no reused QR pass — was only truly confirmed once a test was written that actively tried to break it, rather than one that only confirmed the happy path.

Bibliography

- Festify — Semester Project Proposal, Thapar Institute of Engineering and Technology, 2026.
- FestFlow — Complete Reference: Pitch, Database Design, Architecture & Glossary, compiled August 2026.
- Festify — Technical Codebase Audit & Project Status Report, September 2026.
- IEEE Std 830-1998 — IEEE Recommended Practice for Software Requirements Specifications (structural reference).